INFORMATIONSBLATT

IB02 – UPDATE und DELETE

SQL · LS03 – Daten pflegen (CRUD) · IB02 | 2BKI2 · Schuljahr 2026/2027

🎯 LERNZIEL

Nach diesem Informationsblatt können Sie …

- Datensätze mit UPDATE … SET … WHERE gezielt ändern,
- Datensätze mit DELETE FROM … WHERE gezielt löschen,
- die Sicherheitsregel „erst SELECT, dann UPDATE/DELETE“ anwenden,
- erklären, warum die Datenbank manche Löschungen verweigert, und eine Sicherung anlegen.

1 · Datensätze ändern: UPDATE

```
UPDATE tabelle
SET spalte1 = wert1, spalte2 = wert2
WHERE bedingung;
```

`WHERE` legt fest, **welche** Zeilen geändert werden – meist über den Primärschlüssel. Mehrere Änderungen trennt man hinter SET durch Kommas.

```
UPDATE kunde
SET PLZ = '79104', Ort = 'Freiburg'
WHERE KundenNr = 11;
```

✓ 1 Datensatz betroffen.

Man kann auch mit dem alten Wert rechnen – hier werden alle Citybikes (TypNr 1) um 1 € teurer:

```
UPDATE fahrradtyp SET Tagespreis = Tagespreis + 1.00 WHERE TypNr = 1;
```

✓ 1 Datensatz betroffen.

2 · Das fehlende WHERE

Ohne WHERE gilt das UPDATE für **jede** Zeile der Tabelle. Aus einem Umzug wird ein Massenumzug:

```
UPDATE kunde SET Ort = 'Freiburg';
```

✓ 8 Datensatze betroffen.

```sql
UPDATE kunde SET Ort = 'Freiburg';
SELECT KundenNr, Nachname, PLZ, Ort FROM kunde;
```

▶ **ERGEBNIS**

KundenNr	Nachname	PLZ	Ort
1	Meier	79098	Freiburg
2	Braun	79312	Freiburg
3	Sauer	79098	Freiburg
4	Wolf	79111	Freiburg
5	Keller	79183	Freiburg
… 10 weitere Zeilen …			

15 Zeilen

> ⚠️ **ES GIBT KEIN „RÜCKGÄNGIG“**
>
> phpMyAdmin führt ein UPDATE ohne WHERE ohne Rückfrage aus. Die alten Orte sind weg – nur eine Sicherung (§5) oder ein neuer Import hilft.

> ✅ **DIE GOLDENE REGEL: ERST SELECT, DANN ÄNDERN**
>
> 1. Schreiben Sie zuerst ein `SELECT * FROM … WHERE …` und prüfen Sie, ob **genau** die richtigen Zeilen kommen.
> 2. Ersetzen Sie dann `SELECT * FROM` durch `UPDATE … SET …` bzw. `DELETE FROM` – die WHERE-Bedingung bleibt gleich.
> 3. Vergleichen Sie die Meldung „*n* Datensätze betroffen“ mit der Zeilenzahl aus Schritt 1.

3 · Datensätze löschen: DELETE

```sql
DELETE FROM tabelle WHERE bedingung;
```

```sql
DELETE FROM kunde WHERE KundenNr = 15;
```

✓ 1 Datensatz betroffen.

Auch hier gilt: Ohne WHERE sind **alle** Zeilen weg – die Tabelle selbst bleibt aber bestehen (anders als bei `DROP TABLE`). phpMyAdmin fragt bei DELETE zur Sicherheit nach: *„Möchten Sie die Abfrage … wirklich ausführen?“*

4 · Wenn die Datenbank das Löschen verweigert

Anna Meier hat mehrere Ausleihen. Würde man sie löschen, zeigten diese Ausleihen auf eine Kundin, die es nicht mehr gibt. Der Fremdschlüssel verhindert das:

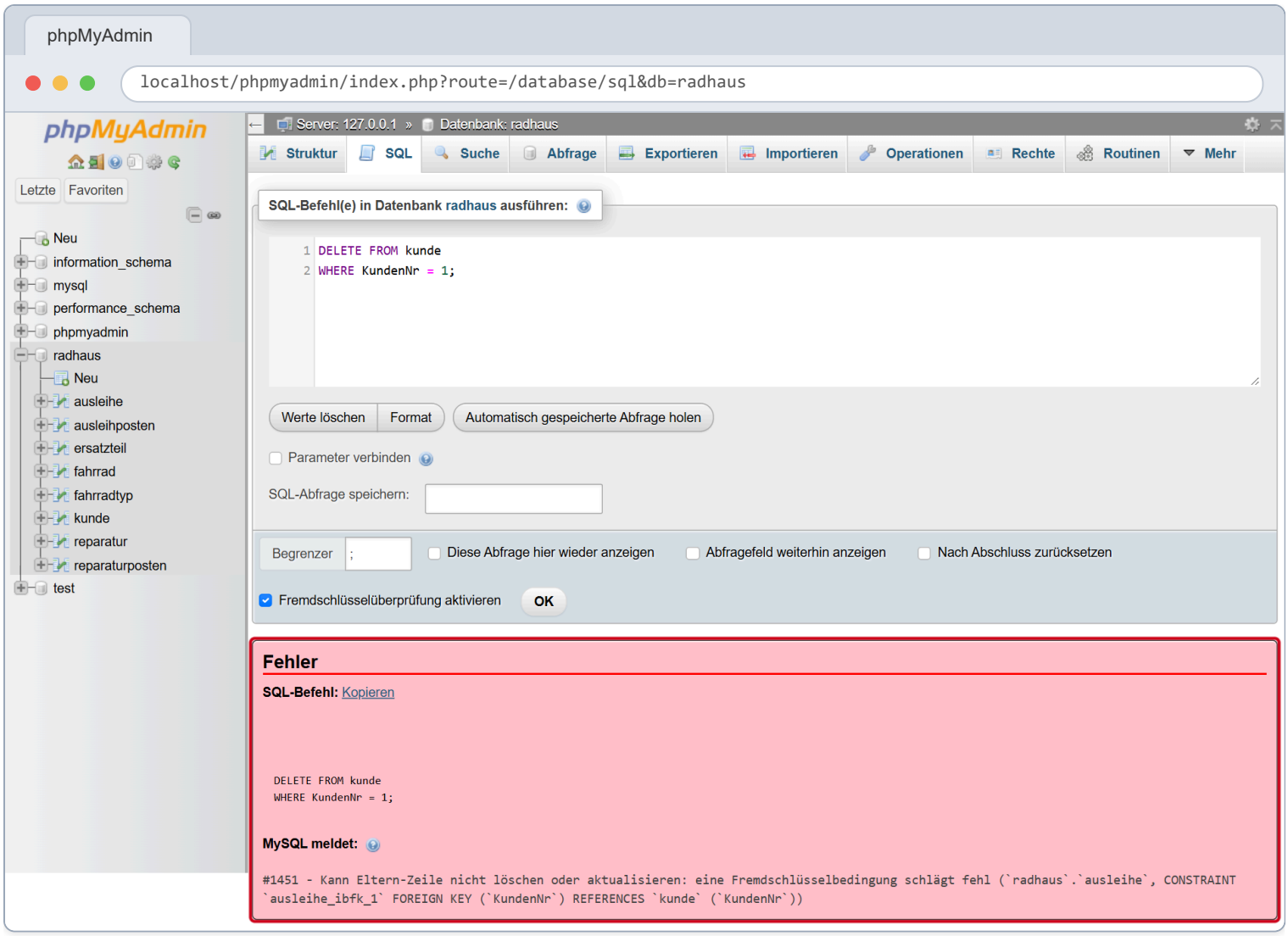


Abb. 1 – Fehler #1451: Eine „Eltern-Zeile“ (die Kundin) kann nicht gelöscht werden, solange „Kind-Zeilen“ (Ausleihen) auf sie verweisen.

Möglichkeit	Bewertung
erst die Ausleihposten, dann die Ausleihen, dann die Kundin löschen	technisch möglich – aber die Umsätze verschwinden aus der Buchhaltung
Kundin behalten, persönliche Daten unkenntlich machen (UPDATE)	in der Praxis üblich: Ausleihen bleiben für die Auswertung erhalten (Datenschutz: „anonymisieren“)

5 · Vorher sichern: Export

Vor größeren Änderungen legt man eine **Sicherung** an: Datenbank links anklicken → Reiter *Exportieren* 1 → *Exportieren* 2 . Sie erhalten eine `.sql`-Datei – genau so eine, wie Sie sie in LS01 importiert haben.

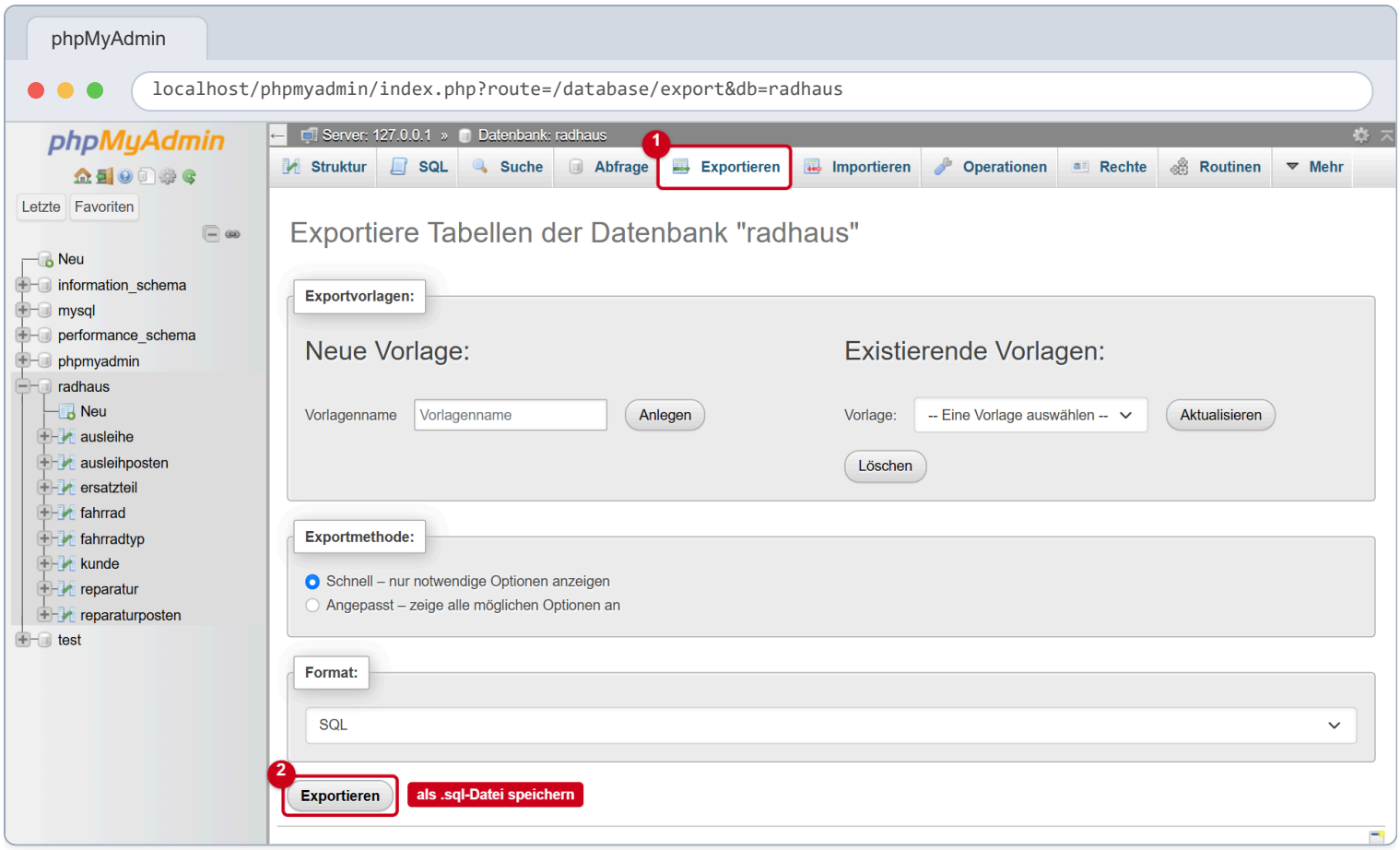


Abb. 2 – Reiter „Exportieren“ – die Schnellmethode mit Format SQL genügt.

> **WIEDERHERSTELLEN**
>
> Eine exportierte Datei spielen Sie über *Importieren* wieder ein (IB01 aus LS01). Achtung: Ein normaler Export enthält kein `DROP DATABASE` – vorher die Tabellen löschen oder eine neue, leere Datenbank wählen.